

占城大师 - 开发方式与项目结构

生成日期：2026-07-05 | 仓库：zhanchengdashi

零、当前项目流程确认

- 上游流程使用公共 codex-game-studio-general-game-development-process.md。
- 公共流程路径：C:\Users\76398\Documents\Codex\2026-07-03\codex-game-studio-default\outputs\code x-game-studio-general-game-development-process.md。
- 本文档是 zhanchengdashi 的项目适配层，负责目录、Godot/GDScript、配置表、验证脚本和 GitHub 同步。
- 当前项目阶段位于 Prototype / Vertical Slice 之间，已经有 Godot 可运行原型和基础闭环。
- 公共流程中的 Engine Specialist 在本项目映射为 Godot Specialist，Language Specialist 映射为 GDScript Specialist。
- 以后所有玩法、数值、UI、系统或技术结构修改，都必须先更新对应设计/流程文档，再实装到游戏中。

一、开发原则

- 所有任务默认按游戏开发任务处理，采用制作、玩法、技术、美术/UI、Godot、GDScript、QA 的角色视角。
- 玩法数值优先数据驱动，设计源表放在 config/tables/，运行时 JSON 放在 runtime/config/。
- 文档是实现前置条件：先在 design/ 或 docs/ 中记录规则、数值、流程、界面或技术决策，再修改配置、脚本、场景和资源。
- 原型表现优先使用 Tween、缩放、位移、闪烁、粒子、材质调色和 UI 弹跳等程序化反馈。
- 每次完成修改都必须提交 Git，并同步到 GitHub 远端。

二、标准任务流程

- 读上游：确认公共流程中对应的角色路由、专项流程和 Definition of Done。
- 读项目：先看 AGENTS.md、本文档、相关 docs、当前文件和 git status。
- 定职责：按任务类型套用最小必要角色组合，例如 UI 走 Art Director、UI Programmer、Godot Specialist、QA Lead。
- 文档先行：玩法、数值、UI、系统或技术结构改动，先写入 design/ 或 docs/，必要时生成 PDF。
- 定落点：再判断改配置、脚本、场景、资源或工具。
- 小步实现：按已经更新的文档实装，保持提交范围聚焦，沿用现有脚本、绘制和数据结构。
- 本地验证：按改动类型运行检查，Godot 脚本改动必须启动项目确认无解析错误。
- 提交同步：git add、git commit、git push origin main。

三、公共流程阶段映射

公共阶段	本项目状态	本项目执行方式
0. Project Startup	已完成	Git、README、目录、Godot 入口、配置表基础已建立。
1. Concept	已有初版	方向是移动端卡牌加占地战斗原型，继续在 docs/CURRENT_GAME_DESIGN.md 沉淀。
2. Prototype	进行中	用 Godot 快速验证战斗、抽卡、编组、升级、地块和 UI。
3. System Design	进行中	规则和数值优先进入文档与配置表，脚本只做运行实现。
3A. Data Config	已建立并持续扩展	config/tables -> validate_config.py -> export_config.py -> runtime/config。
4. Technical Architecture	需要持续补强	main.gd 复杂度上升后拆分到 ui、battle、cards、config。
5. Vertical Slice	进行中	打通大厅、编组、抽卡、战斗、结算、成长完整闭环。
9. Version Finish	每次提交执行	验证通过、commit、push、记录 hash。

四、目录职责

路径	职责	使用规则
design/	设计文档	玩法、数值、UI、系统改动先在这里记录，再进入实现。
config/tables/	设计源表	卡牌、单位、经济、关卡、掉落、地块等可配置内容优先放这里。
runtime/config/	运行时配置	由导出工具生成，只有作为引擎读取源时才提交。
scripts/	Godot 脚本	放运行时代码、原型逻辑、UI 绘制、配置读取。
assets/	源资产	美术、音频、UI、卡牌图、建筑图和后续特效资源。
tools/	开发工具	校验、导出、文档生成、批处理和 QA 辅助脚本。
docs/	可读文档	设计说明、流程、UI/UX、平衡、结构约定。
output/pdf/	审阅版文档	文档 PDF 输出，便于查看和归档。

五、验证门禁

- 任意 .gd 修改后运行 tools/check_gd_indentation.py，并启动 Godot 项目确认没有 parser error。
- 任意配置表修改后运行 tools/validate_config.py；如需更新运行时 JSON，再运行 tools/export_config.py。
- Windows Godot 出现应用程序错误、内存不能为 read 或启动即崩溃时，优先检查渲染后端；本项目默认使用 Vulkan，不强制 D3D12。
- config/tables 下的 CSV 必须保持 UTF-8 或 UTF-8 BOM 编码；不要提交 GBK/ANSI 表格，Unicode decode 错误需先转码再导出。
- 任意文档交付后生成 PDF 到 output/pdf/，并渲染检查页面可读、无重叠、无截断。
- 提交前运行 git diff --check，推送前确认 git status --short --branch。

六、后续结构演进方向

- 当 scripts/app/main.gd 继续增长时，优先拆出 scripts/app/ui/、battle/、cards/、config/。
- 将硬编码原型参数逐步迁移到 config/tables/，保留脚本常量只作为临时桥接。
- 为核心规则补 tests/：卡牌升级、抽卡概率、地块生成、战斗结算和配置导出。
- UI 反复修改后沉淀 docs/UI_COMPONENT_GUIDE.md，形成可复用规范。